CRIPTOGRAFÍA POST-CUÁNTICA

XMSS

eXtended Merkle Signature Scheme

Marco teórico y ejemplo práctico paso a paso

Resistente a computación cuántica · Basado en funciones de hash · Estándar NIST 2022

¿Por qué ECDSA ya no es suficiente?

ECDSA (hoy)	Computadora cuántica
Se basa en el problema del logaritmo discreto sobre curvas elípticas.	El algoritmo de Shor resuelve el logaritmo discreto en tiempo polinómico.
Una computadora clásica necesita ~10^{300} operaciones para romperlo.	Una computadora cuántica de ~4.000 qubits lógicos rompería ECDSA.
Usado en Bitcoin, Ethereum, TLS, SSH, GPG y casi todo internet.	Google, IBM y China están avanzando rápido. El NIST ya se preocupó en 2016.

VS

⚠ La solución: basar la seguridad en funciones de HASH, que los algoritmos cuánticos conocidos NO pueden romper eficientemente.

XMSS — Los 3 pilares

XMSS combina tres ideas para crear un esquema de firma resistente a cuántica:

01

OTS — One Time Signature (Winternitz)

Una clave que firma UN SOLO mensaje. Si se reutiliza, la seguridad se rompe.

02

Árbol de Merkle

Organiza miles de claves OTS en una estructura de árbol con una única clave pública raíz.

03

Estado (Stateful)

XMSS lleva la cuenta de qué claves OTS ya usó. Nunca puede repetir un índice.

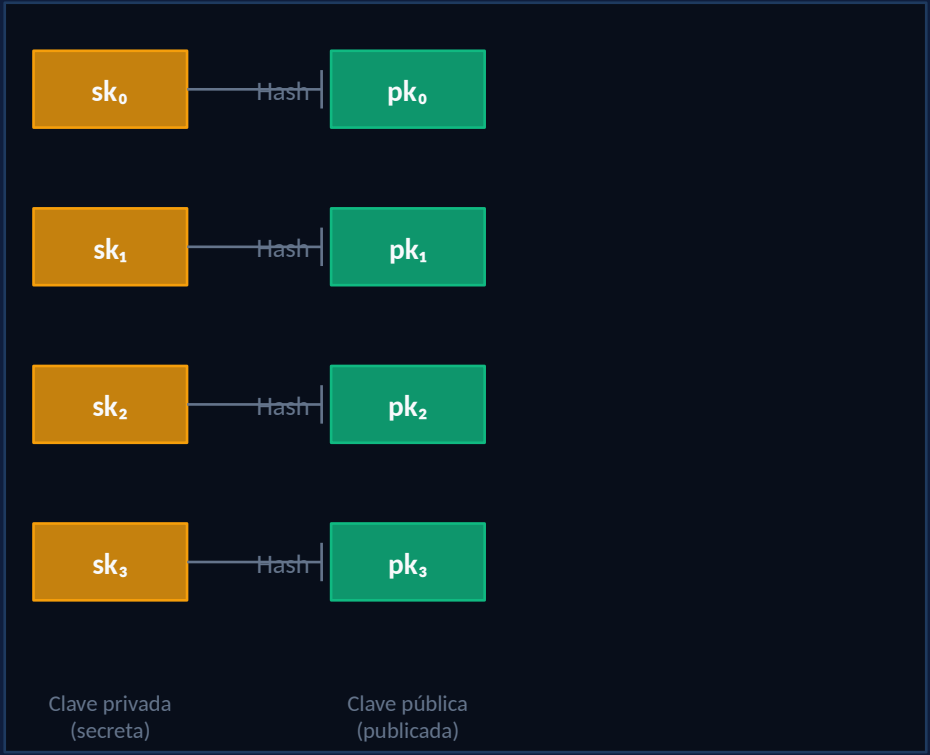
Base matemática: las funciones de hash SHA-256 son resistentes al algoritmo de Grover (cuántica) con pérdida de solo 50% de bits de seguridad — manejable duplicando el tamaño.

Pilar 01 — OTS: Winternitz One-Time Signature

¿Cómo funciona?

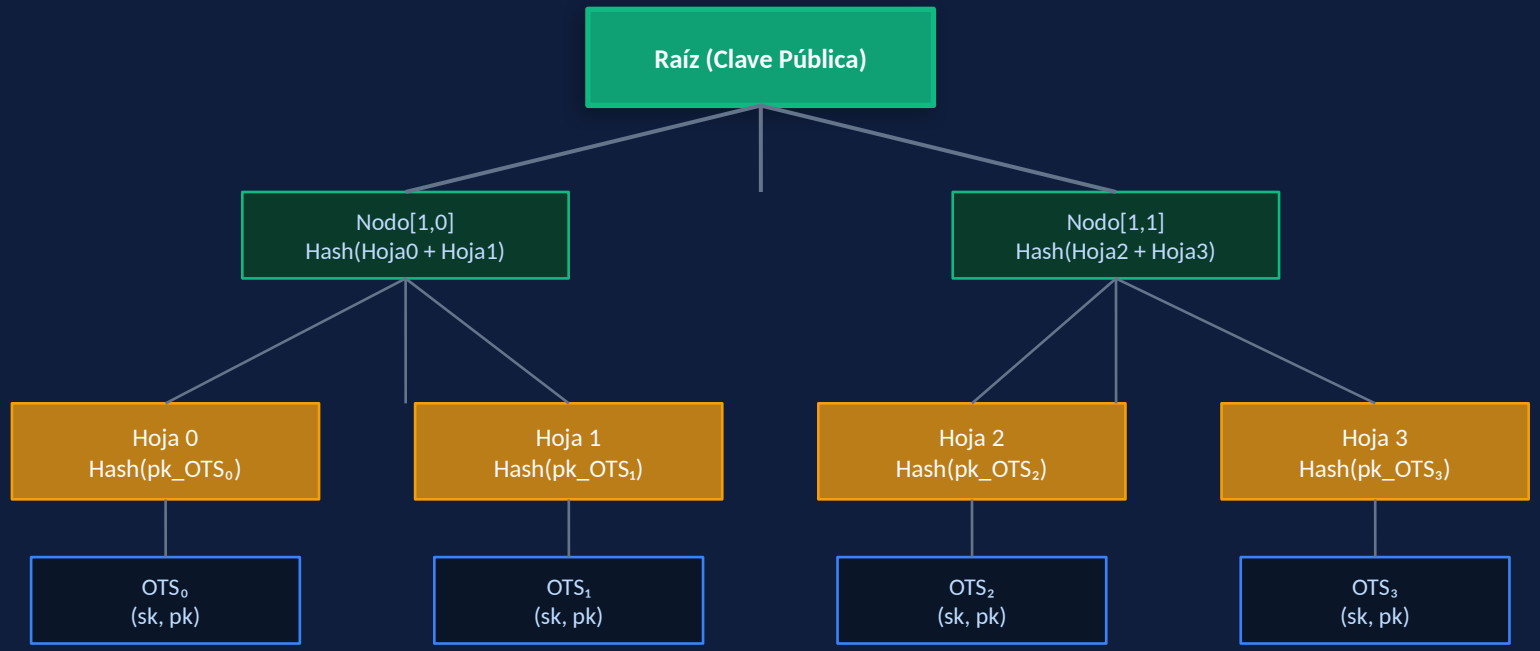
Clave privada	Serie de valores aleatorios: sk_0, sk_1, ... sk_n
Clave pública	Hash de cada valor privado: pk_i = Hash(sk_i)
Firmar	Para cada bit del mensaje, revelás sk_i (bit=1) o su hash (bit=0)
Verificar	El verificador comprueba que los valores revelados hasheados baten con la clave pública
⚠ Solo UNA vez	Si firmás dos mensajes distintos, un atacante puede reconstruir la clave privada combinando ambas firmas

Diagrama simplificado



Pilar 02 — Árbol de Merkle

Tenemos miles de claves OTS de un solo uso. El árbol de Merkle las organiza bajo UNA SOLA clave pública raíz.



La Raíz del árbol = Clave Pública XMSS. Para probar validez de una firma se revela el 'camino de autenticación' desde la hoja hasta la raíz (unos pocos hashes, no todo el árbol).

Pilar 03 — Estado (Stateful)

XMSS es stateful: debe recordar qué índice OTS usó para nunca repetirlo. Esto lo diferencia de ECDSA (que es stateless).

El índice como contador

- 0 idx = 0 → Usar OTS_0
- 1 idx = 1 → Usar OTS_1
- 2 idx = 2 → Usar OTS_2 ← ahora
- 3 idx = 3 → Usar OTS_3

Riesgos del estado

Pérdida del estado

Si se pierde qué índice se usó, no se puede continuar firmando de forma segura.

Backup y restore

Restaurar un backup antiguo puede hacer que se reutilice un índice — error crítico.

Límite de firmas

Un árbol de altura h=20 permite $2^{20} \approx 1$ millón de firmas. Luego hay que generar nueva clave.

Uso ideal

Dispositivos con pocas firmas y almacenamiento confiable: HSMs, firmware, PKI.

EJEMPLO PRÁCTICO

Construyendo XMSS con h=2 (4 hojas)

Vamos a construir un esquema XMSS completo con un árbol de altura 2, lo que nos da exactamente 4 claves OTS (= 4 firmas posibles). Ideal para entender el flujo.

1

Generar 4 pares de claves OTS

Para cada hoja del árbol generamos un par (sk_i, pk_i) usando Winternitz.

2

Construir el árbol de Merkle

Hasheamos las pk_i para obtener las hojas, luego construimos el árbol de abajo hacia arriba.

3

Firmar el primer mensaje

Usamos OTS_0 (índice 0), revelamos la clave privada parcial + el camino de autenticación.

4

Verificar la firma

El verificador recalcula el árbol desde la hoja firmada y comprueba que llega a la misma Raíz.

1 Generar los 4 pares de claves OTS (Winternitz)

Para cada una de las 4 hojas del árbol generamos un par de claves OTS independiente:

OTS0	sk_0 = [a1f3..., b9c2..., e4d7..., ...] sk → PRIVADA, nunca se publica	pk_0 = [Hash(a1f3...), Hash(b9c2...), ...] pk → PÚBLICA, se pone en las hojas del árbol
OTS1	sk_1 = [7f2a..., 3d8e..., c1b4..., ...] sk → PRIVADA, nunca se publica	pk_1 = [Hash(7f2a...), Hash(3d8e...), ...] pk → PÚBLICA, se pone en las hojas del árbol
OTS2	sk_2 = [9e1c..., 4a7d..., 2f8b..., ...] sk → PRIVADA, nunca se publica	pk_2 = [Hash(9e1c...), Hash(4a7d...), ...] pk → PÚBLICA, se pone en las hojas del árbol
OTS3	sk_3 = [6b5f..., 1e9c..., 8a3d..., ...] sk → PRIVADA, nunca se publica	pk_3 = [Hash(6b5f...), Hash(1e9c...), ...] pk → PÚBLICA, se pone en las hojas del árbol

⚡ Cada par OTS es completamente independiente. La seguridad no depende de que estén relacionados entre sí.

2 Construir el árbol de Merkle (de abajo hacia arriba)

1 Nivel 0 — Las hojas

Hoja[0] = Hash(pk_0)
Hoja[1] = Hash(pk_1)
Hoja[2] = Hash(pk_2)
Hoja[3] = Hash(pk_3)

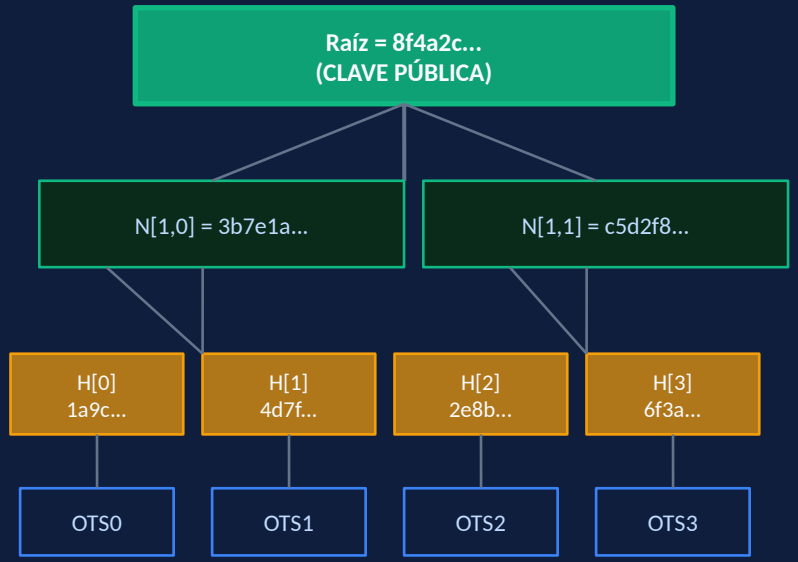
2 Nivel 1 — Nodos intermedios

Nodo[1,0] = Hash(Hoja[0] ‖ Hoja[1])
Nodo[1,1] = Hash(Hoja[2] ‖ Hoja[3])

3 Nivel 2 — La raíz (clave pública)

Raíz = Hash(Nodo[1,0] ‖ Nodo[1,1])
→ Esta Raíz ES la clave pública XMSS

Árbol completo resultante



3 Firmar el primer mensaje — usando OTS_0

Mensaje a firmar: "Transfiero 100 ETH a 0x1234...abcd" → m = Hash(mensaje) = 9d3e4f...

La firma XMSS tiene 3 componentes:

A

Índice

idx = 0
→ Indica que se usó OTS_0

💡 El receptor sabe qué hoja buscar en el árbol

B

Firma OTS

σ_OTS = Winternitz.Sign(sk_0, m)
→ Revela parcialmente sk_0 según los bits de m

💡 Nunca se revela sk_0 completa — solo lo necesario para este mensaje. Ver ejemplo detallado en las próximas 4 slides →

C

Camino de autenticación (Auth Path)

auth = [Hoja[1], Nodo[1,1]]
→ Los 'hermanos' del camino desde OTS_0 hasta la Raíz

💡 Con estos 2 hashes el verificador puede reconstruir la Raíz sin conocer el árbol completo

Estado actualizado: idx avanza de 0 → 1. OTS_0 nunca más se puede usar.

Ejemplo: Winternitz OTS con 4 bits

Paso 1 — Clave privada, cadenas de hash y clave pública

Clave privada (4 valores aleatorios)

$sk_0 = [\ s_1\ ,\ s_2\ ,\ s_3\ ,\ s_4\ ]$

Cadena de hash (w=2, un paso por valor)

$s_1 \rightarrow Hash(s_1) = p_1$

$s_2 \rightarrow Hash(s_2) = p_2$

$s_3 \rightarrow Hash(s_3) = p_3$

$s_4 \rightarrow Hash(s_4) = p_4$

Clave pública de esta OTS

$pk_0 = [\ p_1\ ,\ p_2\ ,\ p_3\ ,\ p_4\ ]$

Cada OTS tiene su propia sk y pk. La pk se hashea para formar una hoja del árbol de Merkle.

Ejemplo: Winternitz OTS con 4 bits

Paso 2 — Firmar: qué se revela según cada bit del mensaje

Hash del mensaje m = 1 0 1 1 (4 bits)

Regla de firma

Bit = 0 → revelar secreto (0 pasos): **s**

Bit = 1 → revelar hash (1 paso): **Hash(s)**

Aplicando al mensaje 1 0 1 1

Bit1=1 → Hash(s_1) = **p_1**

Bit2=0 → **s_2**

Secreto		Hash		Público	Qué se revela
s_1	— Hash →			p_1	bit=1 → revelo **p_1**
s_2	— Hash →			p_2	bit=0 → revelo **s_2**
s_3	— Hash →			p_3	bit=1 → revelo **p_3**
s_4	— Hash →			p_4	bit=1 → revelo **p_4**

Firma = [p_1 , s_2 , p_3 , p_4]

Notar: s_2 (amarillo) es el secreto puro. Los demás (verde) ya son hashes = valores públicos.

Ejemplo: Winternitz OTS con 4 bits

Paso 3 — Verificación: el receptor completa las cadenas de hash

El verificador recibe la firma [p_1 , s_2 , p_3 , p_4] y conoce los bits: **1 0 1 1**
Para cada valor, aplica los pasos de hash que faltan para llegar a la clave pública:

Bit 1 = 1: recibió p_1 → necesita 0 pasos más → ya es p_1 ✓

Bit 2 = 0: recibió s_2 → aplica 1 Hash → Hash(s_2) = p_2 ✓

Bit 3 = 1: recibió p_3 → necesita 0 pasos más → ya es p_3 ✓

Bit 4 = 1: recibió p_4 → necesita 0 pasos más → ya es p_4 ✓

Todos coinciden con $pk_0 = [p_1, p_2, p_3, p_4]$ → FIRMA VÁLIDA ✓

El verificador NUNCA vio los secretos s_1, s_3, s_4. Solo necesitó completar la cadena de s_2 (el único bit=0).
Si un solo valor no coincide, la firma se rechaza inmediatamente.

La seguridad depende de que Hash es irreversible: de p_1 no se puede obtener s_1.

Ejemplo: Winternitz OTS con 4 bits

Paso 4 — ¿Por qué solo UNA vez? El peligro de reutilizar

Firma del mensaje 1 (bits: 1 0 1 1)

$\text{Firma}_1 = [\ p_1\ ,\ s_2\ ,\ p_3\ ,\ p_4\ ]$

Firma del mensaje 2 (bits: 0 1 1 0)

$\text{Firma}_2 = [\ s_1\ ,\ p_2\ ,\ p_3\ ,\ s_4\ ]$

El atacante que ve AMBAS firmas obtiene:

Posición 1: s_1 (de Firma_2) + p_1 (de Firma_1) → **TIENE TODO**

Posición 2: s_2 (de Firma_1) + p_2 (de Firma_2) → **TIENE TODO**

Posición 3: p_3 (de ambas) → no obtiene s_3 → OK (pero innecesario para bit=1)

Posición 4: s_4 (de Firma_2) + p_4 (de Firma_1) → **TIENE TODO**

Con s_1, s_2, s_4 y p_3 puede forjar firmas para CUALQUIER mensaje. Seguridad ROTA.

Por eso XMSS lleva un contador de índice: cada OTS se usa exactamente UNA vez y se descarta.
Esto conecta con el Pilar 03 (Estado): el idx DEBE avanzar después de cada firma.

Del ejemplo al mundo real

El mismo mecanismo Winternitz, pero con parámetros de producción

En la vida real, el mensaje a firmar es algo como: `"Transfiero 100 ETH a 0x7f8B...d4e2"`
Se hashea con SHA-256 → **256 bits** (en nuestro ejemplo usamos solo 4)

Parámetro	Nuestro ejemplo	XMSS real (NIST SP 800-208)
Hash del mensaje	4 bits	256 bits (SHA-256)
Parámetro Winternitz (w)	w=2 (1 bit por trozo)	w=16 (4 bits por trozo)
Trozos del hash	4 trozos de 1 bit	64 trozos de 4 bits
Pasos de hash por cadena	1 paso (0 ó 1)	hasta 15 pasos (0 a 15)
Valores secretos en sk	4	~67 valores

El principio es idéntico: para cada trozo del hash, se revela el secreto hasheado un número de veces que depende del valor de ese trozo. Más trozos y cadenas más largas, pero la misma lógica.

El trade-off de w: mayor w = firma más compacta pero más lenta (más hashes por cadena). w=16 es el balance estándar del NIST.

4 Verificar la firma

El verificador tiene: el mensaje, la firma (idx + σ_OTS + auth_path) y la Raíz (clave pública XMSS).

1 Recalcular pk_OTS desde σ_OTS

Con la firma OTS y el mensaje, el verificador recalcula la clave pública OTS_0 usando las propiedades de Winternitz.

2 Calcular Hoja[0]

Hoja[0] = Hash(pk_OTS_0 recalculada)

3 Subir por el árbol con auth_path

Nodo[1,0] = Hash(Hoja[0] ‖ auth[0]) donde auth[0] = Hoja[1]
Raíz' = Hash(Nodo[1,0] ‖ auth[1]) donde auth[1] = Nodo[1,1]

4 Comparar raíces

Si Raíz' == Raíz (clave pública) → ✅ FIRMA VÁLIDA
Si Raíz' ≠ Raíz → ❌ FIRMA INVÁLIDA

🔑 Clave: el verificador NUNCA necesita ver sk_0. Solo usa pk_0 (recalculada), los hashes del auth_path y la Raíz pública. Seguro y verificable.

ECDSA vs XMSS — Comparación

Característica	ECDSA	XMSS
Base matemática	Curvas elípticas (DLP)	Funciones de hash (SHA-256)
Resistente a cuántica	❌ NO (Algoritmo de Shor)	✅ SÍ
Stateful	No (sin límite de firmas)	Sí (hay que llevar índice)
Tamaño de firma	~64 bytes (muy compacto)	~2-3 KB (mucho mayor)
Tamaño clave pública	33-65 bytes	~64 bytes (solo la raíz)
Velocidad de firma	Muy rápida	Lenta (construye árbol)
Velocidad verificación	Rápida	Rápida (solo auth_path)
Estándar NIST	Sí (FIPS 186)	Sí (SP 800-208, 2022)
Uso ideal	Internet en general	Firmware, PKI, bajo volumen

Lean Ethereum y la resistencia cuántica

Vitalik Buterin presentó en DevConnect Buenos Aires (nov. 2024) el framework 'Lean Ethereum' — un rediseño de la capa base de Ethereum enfocado en simplicidad y seguridad post-cuántica.

Reemplazar BLS por firmas basadas en hash

Las firmas BLS actuales usan curvas elípticas (vulnerables a cuántica). Vitalik propone variantes Winternitz — la misma familia que XMSS.

STARKs para agregación de firmas

Para compensar el tamaño mayor de las firmas hash, se usarán STARKs (pruebas de conocimiento cero) para agregar miles de firmas eficientemente. Los STARKs también son resistentes a cuántica.

Beam Chain — rediseño del consenso

El Beam Chain reduce el tiempo de finalización a 12 segundos y simplifica la estructura para que la transición a primitivas post-cuánticas sea viable sin sacrificar performance.

Conexión directa con XMSS

XMSS y sus variantes Winternitz son parte de la familia de esquemas que el NIST estandarizó (SP 800-208). Ethereum no usará XMSS textualmente pero sí la misma base matemática.

Conclusiones

XMSS se basa en hash

La única suposición de seguridad es la resistencia de SHA-256 — mucho más sólida que el problema del logaritmo discreto.

El árbol de Merkle es el corazón

Permite usar miles de claves OTS de un solo uso bajo una única clave pública de solo 64 bytes.

El estado es crítico

La gestión del índice es la responsabilidad operativa central. Una falla aquí puede comprometer toda la seguridad.

El futuro es post-cuántico

El NIST lo estandarizó en 2022. Lean Ethereum lo está adoptando. Quien diseña sistemas hoy debe considerarlo.